22/06/2026

# Medallion architecture: what it is, how it works and how to implement it without mistakes

There's a question every data team ends up asking at some point: **how do we organize the data inside the lakehouse so it's reliable, maintainable and useful for both analytics and machine learning**? The Medallion architecture is the answer that has become the industry standard. Not because it's the only way to structure a lakehouse, but because it solves a specific problem systematically: making data improve progressively in quality as it moves through the system, with clear responsibilities at each stage and without anyone having to rebuild from scratch when something fails.

This article explains what it is, how each layer works, what design decisions you have to make before implementing it, and what mistakes lead experienced teams to end up with a Medallion architecture that

doesn't work as it should.

# What the Medallion architecture is

The Medallion architecture is a **data design pattern used to organize data logically in a lakehouse**, with the aim of incrementally and progressively improving the structure and quality of the data as it flows through each layer of the architecture (from Bronze ⇒ Silver ⇒ Gold layer tables).

The name comes from medals: **bronze, silver and gold**. Each layer represents a higher level of refinement than the previous one. **Data enters raw, is processed and validated in silver, and comes out ready for consumption in gold**. Medallion architectures are sometimes also called "multi-hop" architectures: each layer is a hop that adds quality and structure.

What makes this pattern especially valuable isn't its simplicity, but what



is responsible for Silver being reliable for analysis. Domain owners are responsible for Gold meeting the business requirements. Without that clarity of responsibilities, data quality problems become entrenched because no one knows exactly at what point in the flow they occurred.

Databricks proposed the pattern in its official documentation, and since then it has become the de facto standard for implementations both in Databricks and in **Microsoft Fabric**, where Microsoft has adopted the same model with the same terminology.

# The three layers: Bronze, Silver and Gold

## Bronze layer: the immutable source of truth

The Bronze layer acts as the initial landing zone for all incoming raw data. Data arrives exactly as it comes from the sources: no transformations, no cleaning, no validations. A record that arrives

duplicated from the ERP is stored duplicated in Bronze. A NULL value coming from the sales system is saved as NULL. A field with an inconsistent date format between two sources is stored with that inconsistency.

That isn't a design flaw: it's precisely the goal. **Bronze is the layer that guarantees you can always go back to the origin**. If a downstream transformation turns out to be incorrect (because the business logic was wrong, because the team misinterpreted a requirement, because the source data changed format), you can reprocess from Bronze without going back to the source systems. That reproducibility is what makes Bronze the system's source of truth, not a junk store.

What is added in Bronze is ingestion metadata: load timestamp, source identifier, batch or event identifier. That information is what later makes it possible to trace the lineage of any piece of data back to its exact origin.

## Silver layer: clean, reliable, conformed data

Silver is the layer where the substantial transformation work happens. **The Silver layer brings together data from different sources into an enterprise view and enables self-service analysis for ad hoc reports, advanced analytics and ML**.

In practice, Silver is where the problems Bronze left visible get resolved: duplicates are removed, formats are normalized, ranges and types are validated, basic business rules are applied, and entities that appear under different identifiers in different source systems are consolidated. A customer with three different IDs in CRM, ERP and the e-commerce platform is unified here into a single canonical record.

In the lakehouse data engineering paradigm, the ELT methodology is usually followed instead of ETL, which means only minimal or just-enough transformations are applied while loading the Silver layer. Speed and agility to bring in and deliver data are prioritized. This has an important practical implication: Silver isn't the place for business aggregations or dimensional models. It's the place where data is reliable

and consistent, not where it's optimized for a specific use case.

### Gold layer: data ready for consumption

The Gold layer generates enterprise-level aggregates, star-schema models and feature tables optimized for specific analytics use cases, dashboards, **machine learning** training and serving end-user queries with guaranteed service-level agreements.

Each Gold table is designed for a specific consumer with specific requirements. The management sales dashboard needs data aggregated by week, market and product line. The purchase-propensity model needs a feature table with variables calculated over the last 90-day window. The regulatory report needs a specific view with the exact fields and transformations the regulation requires. Each of these is a different Gold product, even though they all start from the same Silver data.

One consequence of this design is that Gold can and should have multiple tables for different domains and use cases. There's no "the Gold table" for an entity: there are the Gold tables the consumers of that entity need, optimized for their specific access patterns.

## Design decisions no one explains

The theory of the three layers is simple. What isn't simple are the decisions you have to make before and during implementation. These are the ones that most affect whether the architecture ends up working or not.

### What goes in Silver and what goes in Gold?

The boundary between Silver and Gold is the one that causes the most confusion. The most useful rule of thumb: Silver holds data that is true for the whole organization. Gold holds data that is true for a specific use case. Universal business rules (how an active customer is defined, what constitutes a valid transaction) go in Silver. Aggregations, dimensional models and ML features go in Gold.

### One Gold per consumer or shared Golds?

It depends on the level of divergence between the requirements. If two teams need the same data at the same granularity but with different filters, a shared Gold table with query-time filters is enough. If their aggregation, latency or semantic requirements diverge significantly, creating separate Gold products is the right call. Trying to serve consumers with very different requirements from a single Gold table usually ends in monstrous tables with dozens of columns no one fully understands.

### How to handle schema changes in Bronze?

Bronze stores data as it arrives, which means it has to be tolerant of schema changes in the sources. The standard solution in **Delta Lake** and **Databricks** is automatic schema evolution with version logging: when a source adds a new column, Bronze absorbs it and records which version it appeared in. The downstream layers (Silver and Gold) receive the changes in a controlled way, with tests that validate the change doesn't break the existing transformations.

## Medallion in streaming: how the pattern changes with real-time data

The Medallion architecture was originally designed for batch processing. When an organization needs its data to reach Gold in seconds instead of hours, the pattern adapts but the principles don't change.

With streaming tables and materialized views, users can create streaming pipelines built on **Apache Spark** Structured Streaming that update incrementally. In practice, this means Bronze can ingest real-time events from Kafka or Event Hubs, Silver applies the transformations continuously on the stream instead of on batches, and Gold materializes the aggregated views with the latency the use case requires.

In Microsoft Fabric's Real-Time Intelligence, the Bronze layer can ingest data through Eventstream or a table in Eventhouse. This layer provides a

base for the subsequent enrichment and analysis in the Silver and Gold layers.

The critical point in streaming Medallion is the same as in batch but more urgent: quality validations have to happen in Bronze before the data reaches Silver. In batch, a quality error is discovered in the next run and the impact is bounded to that batch. In streaming, a quality error not caught in Bronze can contaminate Silver and Gold within seconds before anyone notices.

## Common implementation mistakes

Knowing the pattern isn't enough to implement it well. These are the mistakes that recur most often, even in experienced teams.

1. **Transforming too much in Bronze.** The most common mistake in teams coming from traditional ETL. The temptation to clean the data while loading it into Bronze is understandable, but it removes the ability to reprocess from the origin. If you transform in Bronze and the transformation was wrong, you can't recover the original data without going back to the sources.
2. **Building Silver for a single consumer.** Silver should be a shared organizational asset. When it's designed to serve exclusively one dashboard or one specific model, it ends up being a Gold disguised as Silver. The problem appears when a second team needs that data and discovers that the transformations in Silver serve the first team's requirements, not theirs.
3. **Not defining domain ownership.** The Medallion architecture isn't a trend of the last decade: it's how production-ready AI is done. Organizations that do it well (with data quality control in Bronze, Silver organized by domains feeding MDM, and Gold products certified for AI) will be best positioned to adopt agentic AI at the platform level. Without clear domain owners, Silver tables become no-man's-land that no one maintains when the sources change.
4. **Ignoring data lineage from the start.** Data lineage (what transformations a piece of data underwent from Bronze to Gold, which versions of the source tables were used) is critical for

auditing, debugging and governance. In Databricks with Unity Catalog, lineage is captured automatically if the pipelines are well configured. Not enabling it from the start means having to reconstruct it retroactively, which is costly and always incomplete.

## Medallion architecture and AI agents: the 2026 standard

The role of the Medallion architecture is evolving. It began as a data organization pattern for analytics. In 2026 it's the foundation on which enterprise AI agents operate.

An agent that has to make a decision (recommend a commercial action, consolidate an operational report, prioritize an incident) needs data that is reliable, with documented lineage and clear semantics. Gold is exactly that. When Microsoft IQ activates Fabric IQ to give context to agents, what it's consuming are the Gold tables of the lakehouse, with the semantics Unity Catalog has documented about them.

This semantic layer is what turns a data platform from a technological asset into a fabric of operational intelligence. It's also where AI agents add enormous value: agents that can traverse the knowledge graph, answer questions and surface insights that structured queries wouldn't reach.

The practical consequence for any organization planning to use AI agents: the state of its Medallion architecture directly determines the quality of what its agents produce. An agent consuming well-designed, certified Gold tables with documented lineage makes correct decisions. An agent consuming Gold tables built on an ungoverned **data swamp** makes incorrect decisions with the same confidence. The Medallion architecture isn't just a good data engineering practice: it's the infrastructure that makes it possible for AI to work reliably in production.

## How BertIA works with the Medallion architecture

At BertIA, the Medallion architecture is the base pattern in any data

project on Databricks or Microsoft Fabric. Not as a template applied mechanically, but as a framework adapted to each client's particularities: data volume, number of sources, latency requirements, team structure and priority use cases.

What we've learned implementing this pattern in sectors such as pharmaceutical, industrial and logistics is that the biggest success factor isn't technical: it's clarity about who owns each data domain. Before creating a single Bronze table, the projects where the deployment works well have defined which team is responsible for each ingestion source, what "correct data" means for each business entity, and what SLA each Gold table has with the consumers that depend on it.

Integration with Databricks' Unity Catalog is a standard part of our implementations: automatic lineage between layers, access control by domain and metadata documentation are the elements that turn a Medallion architecture into an auditable system. For clients in regulated sectors, that traceability is what makes it possible to answer an audit without reconstructing anything manually. For the MLOps projects that manage models in production on this data, certified Gold is the guarantee that training and serving always start from data with verified quality.

## Conclusion

The Medallion architecture has been the standard for structuring data in a data lakehouse for years. In 2026, with agentic AI as the layer that consumes that data to make autonomous decisions, its relevance has multiplied: organizations with a well-implemented Medallion are building on a foundation that will serve them for years to come. Those with a poorly implemented one (no domain ownership, no lineage, transformations in the wrong layer) will have to pay for it the moment they try to scale their AI initiatives.

If you're designing or reviewing your data architecture and want to make sure Medallion is configured to support both current analytics and the AI

use cases coming, **talk to our team** to review your specific case.

Latest posts

# Stay current with BertIA



## BertIA, a partner of Anthropic's Claude Partner Network

16/07/2026

Leer más



## Microsoft Fabric: what it is, what it's for and when it makes sense to adopt it

14/07/2026

Leer más



## Delta Sharing: how to share data across platforms and formats without copying it

25/06/2026

Leer más



## What a data swamp is and how to spot it in your company

15/06/2026

Leer más

We build Data and AI solutions for companies that want real impact.

## Solutions

Artificial Intelligence

Data & Analytics

## About BertIA

Partners

About us

Join the team

## Contact

BARCELONA OFFICE
Pablo Iglesias, 56, 1º 1ª
08302 Mataró, Barcelona

MADRID OFFICE
Glorieta de Quevedo 9
28015 Madrid

+34 931 31 31 83

hola@bertia.es

Copyright © 2026 BUSINESS ANALYTICS PARTNERS, S.L. All rights reserved.

Legal Notice Privacy Policy

Cookie Policy